

ЦОИ — ЛР0 — вариант 5

Частотная фильтрация периодических помех: режекторные фильтры (ring/notch).

```
clear; close all; clc;

thisDir = fileparts(mfilename("fullpath"));
imgDir = fullfile(thisDir, "Рисунки");
outDir = fullfile(thisDir, "Результаты");
if ~exist(outDir, "dir"); mkdir(outDir); end

% Режим синтеза помехи для части 1:
% "diagonal" — одна плоская волна с частотами (u0,v0) (наклонная)
waveMode = "diagonal";
```

Часть 1 — синтез помехи и фильтрация (A6_05_1.jpg)

```
Irgb = imread(fullfile(imgDir, "A6_05_1.jpg"));
I = im2double(rgb2gray(Irgb));
[M, N] = size(I);

% Плоская наклонная волна: 50 периодов по вертикали, 150 по горизонтали
vPeriods = 50;
uPeriods = 150;
P = makePlaneWave(M, N, vPeriods, uPeriods, waveMode);

% Ввод помехи с энергией в 4 раза больше энергии изображения
energyRatio = 4;
a = scaleToEnergyRatio(I, P, energyRatio);
Inoise = I + a * P;

F = fftshift(fft2(Inoise));
specNoisy = spectrumForDisplay(F);

% Частоты помехи в координатах fftshift (центр = 0)
u0 = uPeriods;
v0 = vPeriods;
W = 6;      % ширина вырезаемой полосы вокруг радиуса (ring)
R = 3;      % радиус вырезаемых точек вокруг пиков (notch)

% Фильтры подстраиваем под выбранный тип помехи
switch waveMode
    case {"diagonal", "product"}
        % пики: ( $\pm u_0, \pm v_0$ ), кольцо:  $D_0 = \sqrt{u_0^2 + v_0^2}$ 
        D0 = hypot(u0, v0);
        Hring = idealBandReject(M, N, D0, W);
        notches = [ u0,  v0;
                    -u0, -v0;
                    u0, -v0;
                    -u0,  v0];
```

```

Hnotch = idealNotchReject(M, N, notches, R);

case "sum"
    % пики:  $(\pm u_0, 0)$  и  $(0, \pm v_0)$ , кольца:  $D_0 = u_0$  и  $D_0 = v_0$ 
    Hring = idealBandReject(M, N, u0, W) .* idealBandReject(M, N, v0, W);
    notches = [ u0, 0;
                -u0, 0;
                0, v0;
                0, -v0];
    Hnotch = idealNotchReject(M, N, notches, R);

otherwise
    error("Unknown waveMode: %s", waveMode);
end

Iring = real(ifft2(ifftshift(F .* Hring)));
Inotch = real(ifft2(ifftshift(F .* Hnotch)));
IringDisp = mat2gray(Iring);
InotchDisp = mat2gray(Inotch);

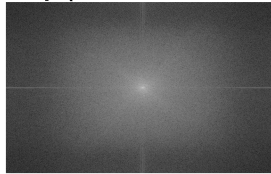
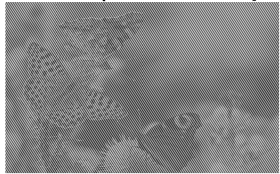
% Оценка (сравнение с исходным I)
mseRing = mean((Iring(:) - I(:)).^2);
mseNotch = mean((Inotch(:) - I(:)).^2);

% Визуализация
fig1 = figure("Name", "Задание 5 – Часть 1", "Color", "w");
tiledlayout(fig1, 2, 3, "Padding", "compact", "TileSpacing", "compact");
nexttile; imshow(I); title("Исходное (полутонное)");
nexttile; imshow(mat2gray(Inoise)); title(sprintf("С помехой (Епом/Еизобр=%.1f)", energyRatio));
nexttile; imshow(specNoisy, []); title("Спектр (лог + адапт. нелин.)");
nexttile; imshow(Hring, []); title("Кольцевой режекторный фильтр");
nexttile; imshow(IringDisp); title(sprintf("Фильтрация кольцом (MSE=%.4g)", mseRing));
nexttile; imshow(InotchDisp); title(sprintf("Фильтрация точками (MSE=%.4g)", mseNotch));

exportgraphics(fig1, fullfile(outDir, "part1_overview.png"), "Resolution", 200);

```

Исходное (полутонное) С помехой (Епом/Еизобр=4.0) Спектр (лог + адапт. нелин.)



Кольцевой режекторный фильтр Фильтрация кольцом (MSE=0.001159) Фильтрация точками (MSE=3.51e-1)



```
imwrite(mat2gray(P), fullfile(outDir, "part1_noise_pattern.png"));
imwrite(mat2gray(Inoise), fullfile(outDir, "part1_noisy.png"));
imwrite(IringDisp, fullfile(outDir, "part1_filtered_ring.png"));
imwrite(InotchDisp, fullfile(outDir, "part1_filtered_notch.png"));
```

Часть 2 — обнаружение частот помех и фильтрация (A6_05_2.jpg)

```
I2 = im2double(imread(fullfile(imgDir, "A6_05_2.jpg")));
if ndims(I2) == 3
    I2 = rgb2gray(I2);
end
[M2, N2] = size(I2);
F2 = fftshift(fft2(I2));
spec2 = spectrumForDisplay(F2);

% Поиск помеховых частот как ярких пиков спектра (кроме центра)
numPeaks = 10;
centerRadius = 12; % игнорируем центр
minSeparation = 8;
peaks = findSpectrumPeaks(F2, numPeaks, centerRadius, minSeparation);

R2 = 2.5;
H2 = idealNotchReject(M2, N2, peaks, R2);
I2f = real(ifft2(ifftshift(F2 .* H2)));
I2f = mat2gray(I2f);
```

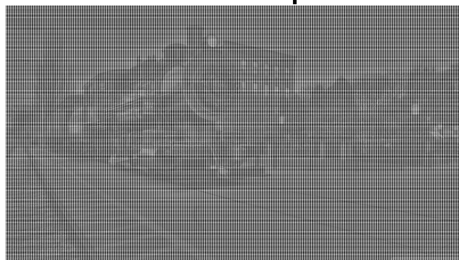
```

fig2 = figure("Name", "Задание 5 – Часть 2", "Color", "w");
tiledlayout(fig2, 2, 2, "Padding", "compact", "TileSpacing", "compact");
nexttile; imshow(I2); title("Искажённое изображение");
nexttile; imshow(spec2, []); title("Спектр (лог + адапт. нелин.)");
nexttile; imshow(H2, []); title("Точечный режекторный фильтр");
nexttile; imshow(I2f); title("Результат фильтрации");

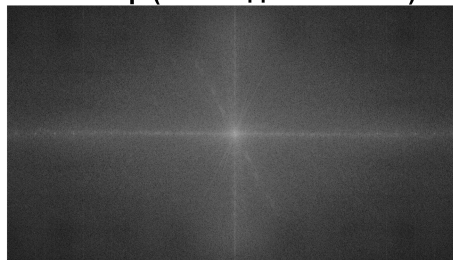
exportgraphics(fig2, fullfile(outDir, "part2_overview.png"), "Resolution", 200);

```

Искажённое изображение



Спектр (лог + адапт. нелин.)



Точечный режекторный фильтр



Результат фильтрации



```

imwrite(I2f, fullfile(outDir, "part2_filtered_notch.png"));

fprintf("Часть 1: MSE ring=%.6g, notch=%.6g\n", mseRing, mseNotch);

```

Часть 1: MSE ring=0.000115861, notch=3.51048e-06

```

fprintf("Часть 2: найдено пиков (с учётом симметрии): %d\n", size(peaks, 1));

```

Часть 2: найдено пиков (с учётом симметрии): 10

Локальные функции

```

function P = makePlaneWave(M, N, vPeriods, uPeriods, mode)
[x, y] = meshgrid(0:N-1, 0:M-1);
switch mode
    case "diagonal"
        P = sin(2*pi*(uPeriods*x/N + vPeriods*y/M));
    case "product"

```

```

        P = sin(2*pi*(uPeriods*x/N)) .* sin(2*pi*(vPeriods*y/M));
    case "sum"
        P = sin(2*pi*(uPeriods*x/N)) + sin(2*pi*(vPeriods*y/M));
    otherwise
        error("Unknown mode: %s", mode);
end
end

function a = scaleToEnergyRatio(I, P, ratio)
Ei = sum(I(:).^2);
Ep = sum(P(:).^2);
a = sqrt((ratio * Ei) / max(Ep, eps));
end

function S = spectrumForDisplay(Fshift)
A = abs(Fshift);
L = log1p(A);
L = L / max(L(:));
gamma = 0.7;
S = L .^ gamma;
end

function H = idealBandReject(M, N, D0, W)
[U, V] = freqGrid(M, N);
D = hypot(U, V);
H = ones(M, N);
H(abs(D - D0) <= (W/2)) = 0;
end

function H = idealNotchReject(M, N, notches, R)
[U, V] = freqGrid(M, N);
H = ones(M, N);
for k = 1:size(notches, 1)
    u0 = notches(k, 1);
    v0 = notches(k, 2);
    Dk = hypot(U - u0, V - v0);
    H(Dk <= R) = 0;
end
end

function [U, V] = freqGrid(M, N)
u = (-floor(N/2)):ceil(N/2)-1;
v = (-floor(M/2)):ceil(M/2)-1;
[U, V] = meshgrid(u, v);
end

function peaks = findSpectrumPeaks(Fshift, numPeaks, centerRadius, minSep)
A = log1p(abs(Fshift));
A = A / max(A(:));
[M, N] = size(A);

```

```

[U, V] = freqGrid(M, N);
mask = hypot(U, V) >= centerRadius;
vals = A(mask);
idxAll = find(mask);
[~, order] = sort(vals, "descend");
idxSorted = idxAll(order);
chosen = zeros(0, 2);
for t = 1:numel(idxSorted)
    if size(chosen, 1) >= numPeaks
        break;
    end
    [r, c] = ind2sub([M, N], idxSorted(t));
    u0 = U(r, c);
    v0 = V(r, c);
    if ~isempty(chosen)
        if any(hypot(chosen(:, 1) - u0, chosen(:, 2) - v0) < minSep)
            continue;
        end
    end
    chosen(end+1, :) = [u0, v0];
end
peaks = unique([chosen; -chosen], "rows");
end

```